TEMCO LEARN

T3000 API & Protocol

REST API · built Nov 13, 2026

REST API

What is the REST API?

The REST API allows you to access T3000 device data through web requests. Think of it like asking questions to your building automation system and getting answers back.

When Would I Use This?

Common Uses:

- View live temperature readings from anywhere
- Check if HVAC equipment is running
- Update setpoints remotely
- Build custom dashboards
- Integrate with other software

How Does It Work?

1. **You send a request** (like "show me all devices")
2. **T3000 processes it** and gathers the information
3. **You receive the data** in an easy-to-read format

Example: Get All Devices

What you want: See a list of all connected devices

How to do it:

```
TEXT

Open your web browser and visit:
http://localhost:9103/api/t3_device/devices
```

What you get back: A list showing each device's name, location, and status.

Getting Started

Requirements:

- T3000 API server must be running
- Know the server address (usually `localhost` if on same computer)
- Use port `9103` (default)

Testing the API:

1. Start the T3000 API server
2. Open a web browser
3. Type in the address bar: `http://localhost:9103/api/t3_device/devices`
4. Press Enter
5. You should see a list of your devices

Common Operations

See all devices:

```
TEXT

http://localhost:9103/api/t3_device/devices
```

View device inputs (sensors):

```
TEXT

http://localhost:9103/api/t3_device/devices/YOUR_DEVICE_ID/inputs
```

View device outputs (controls):

```
TEXT

http://localhost:9103/api/t3_device/devices/YOUR_DEVICE_ID/outputs
```

Replace `YOUR_DEVICE_ID` with your actual device number.

Need Help?

- **Can't connect?** Make sure the API server is running
- **Getting errors?** Check that the device ID is correct
- **Want to do more?** Switch to the "Technical" tab for code examples

REST API Technical Reference

HTTP REST API for T3000 device data access and control.

Base URL

```
TEXT

http://localhost:9103/api
```

API Endpoints

Device Management

List all devices:

```
HTTP

GET /t3_device/devices
```

Response:

```
JSON

[
  {
    "serial": 237219,
    "name": "Controller_01",
    "online": true,
    "product": "T3-BB",
    "address": "192.168.1.100"
  }
]
```

Get device details:

```
HTTP

GET /t3_device/devices/{serial}
```

Get table data:

```
HTTP

GET /t3_device/devices/{serial}/table/{table}
```

Available tables: `INPUTS`, `OUTPUTS`, `VARIABLES`, `PROGRAMS`, `SCHEDULES`, `HOLIDAYS`, `CONTROLLERS`, `SCREENS`, `MONITORS`

Data Points

Get all inputs:

```http
GET /t3_device/devices/{serial}/inputs
```

Response:

```json
{
  "points": [
    {
      "index": 1,
      "label": "Zone_Temp",
      "value": 72.5,
      "units": "DEG_F",
      "range": "0-100",
      "calibration": 0.0,
      "filter": 3,
      "status": "OK"
    }
  ]
}
```

Get all outputs:

```http
GET /t3_device/devices/{serial}/outputs
```

Batch save inputs:

HTTP

```
POST /t3_device/devices/{serial}/batch_save_inputs
Content-Type: application/json

{
  "points": [
    {
      "index": 1,
      "label": "Zone_Temp",
      "value": 72.5,
      "units": "DEG_F"
    }
  ]
}
```

FFI (Foreign Function Interface) Calls

Execute FFI function:

HTTP

```
POST /t3_device/ffi/call
Content-Type: application/json

{
  "action": 17,
  "serial": 237219,
  "panel": 0,
  "parameter": 0
}
```

Common action codes:

- `17` : Read inputs
- `18` : Read outputs
- `19` : Read variables
- `20` : Write output
- `1000` : Scan network

Authentication

Currently no authentication required (development mode only).

Production recommendations:

- Implement API key authentication
- Use OAuth2 for user-based access
- Enable HTTPS/TLS encryption

Response Format

All responses are JSON format with standard HTTP status codes:

Success codes:

- `200 OK`: Request successful
- `201 Created`: Resource created
- `204 No Content`: Success with no response body

Error codes:

- `400 Bad Request`: Invalid parameters
- `404 Not Found`: Resource not found
- `500 Internal Server Error`: Server error
- `503 Service Unavailable`: Database locked or service down

Error response format:

```JSON
{
  "error": "Device not found",
  "code": 404,
  "details": "Serial number 999999 does not exist"
}
```

Rate Limiting

Current limits (development):

- 100 requests per minute per IP
- 10 concurrent connections
- No burst allowance

Exceeding limits:

```JSON
{
  "error": "Rate limit exceeded",
  "code": 429,
  "retry_after": 60
}
```

Code Examples

JavaScript/TypeScript

TYPESCRIPT

```typescript
// Fetch all devices
async function getDevices() {
  const response = await fetch('http://localhost:9103/api/t3_device/devices');
  if (!response.ok) {
    throw new Error(`HTTP error! status: ${response.status}`);
  }
  const devices = await response.json();
  return devices;
}

// Get device inputs
async function getInputs(serial: number) {
  const url = `http://localhost:9103/api/t3_device/devices/${serial}/inputs`;
  const response = await fetch(url);
  const data = await response.json();
  return data.points;
}

// Update input point
async function updateInput(serial: number, index: number, label: string) {
  const url = `http://localhost:9103/api/t3_device/devices/${serial}/batch_save_inputs`;
  const response = await fetch(url, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({
      points: [{ index, label }]
    })
  });
  return response.json();
}
```

Python

PYTHON

```python
import requests

## Get all devices
def get_devices():
    response = requests.get('http://localhost:9103/api/t3_device/devices')
    response.raise_for_status()
    return response.json()

## Get device inputs
def get_inputs(serial):
    url = f'http://localhost:9103/api/t3_device/devices/{serial}/inputs'
    response = requests.get(url)
    response.raise_for_status()
    return response.json()['points']

## Update input
def update_input(serial, index, label):
    url = f'http://localhost:9103/api/t3_device/devices/{serial}/batch_save_inputs'
    payload = {
        'points': [{'index': index, 'label': label}]
    }
    response = requests.post(url, json=payload)
    response.raise_for_status()
    return response.json()
```

cURL

SHELL

```shell
## List devices
curl http://localhost:9103/api/t3_device/devices

## Get inputs
curl http://localhost:9103/api/t3_device/devices/237219/inputs

## Update input (POST)
curl -X POST http://localhost:9103/api/t3_device/devices/237219/batch_save_inputs \
  -H "Content-Type: application/json" \
  -d '{"points":[{"index":1,"label":"Updated_Label"}]}'
```

Best Practices

1. **Error Handling**
    - Always check response status codes
    - Implement retry logic with exponential backoff
    - Log errors for debugging
2. **Performance**
    - Use batch operations when updating multiple points
    - Cache responses appropriately (consider stale-while-revalidate)
    - Implement request debouncing for user input
3. **Security**
    - Use HTTPS in production
    - Validate and sanitize all inputs
    - Implement proper authentication
    - Don't expose API directly to internet
4. **Monitoring**
    - Track API response times
    - Monitor error rates
    - Set up alerts for service degradation

Next Steps

- WebSocket API - Real-time data updates
- Events - Event system documentation
- Modbus Protocol - Modbus integration